

INTRODUCTION TO MACHINE LEARNING

Supervised Learning & Linear Regression

Fit lines by hand, walk the loss landscape, and watch gradient descent learn.

Data

Given a table of numbers, what can you do?

size (m ²)	bedrooms	age (yr)	price (\$1000s)
210	3	21	400
160	3	8	330
240	3	14	369
141	2	32	232
300	4	3	540

- **Visualize it:** look at it!
- **Summarize it:** mean, spread, outliers
- **Clean it:** missing values, errors
- **Predict with it:** today's topic

Inputs and Outputs

INPUTS

x : what we are given
size, bedrooms, age ("features")

OUTPUT

y : what we want to predict
price ("target", "label")

SUPERVISED LEARNING

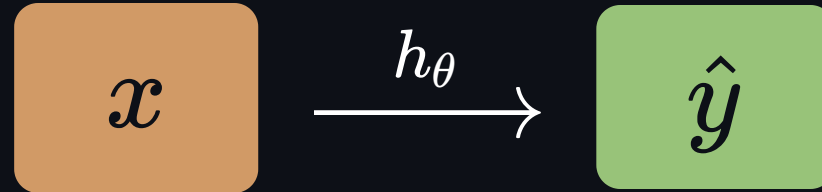
Learn the map from inputs to outputs using example pairs.

$$(x^{(i)}, y^{(i)}) , \quad i = 1, \dots, n$$

Superscript (i) indexes *training examples*;
subscript j indexes *features*: $x_j^{(i)}$. We have n
examples, d features.

The Hypothesis

Given the data, find a function h_θ , the **hypothesis**, that predicts outputs from inputs:



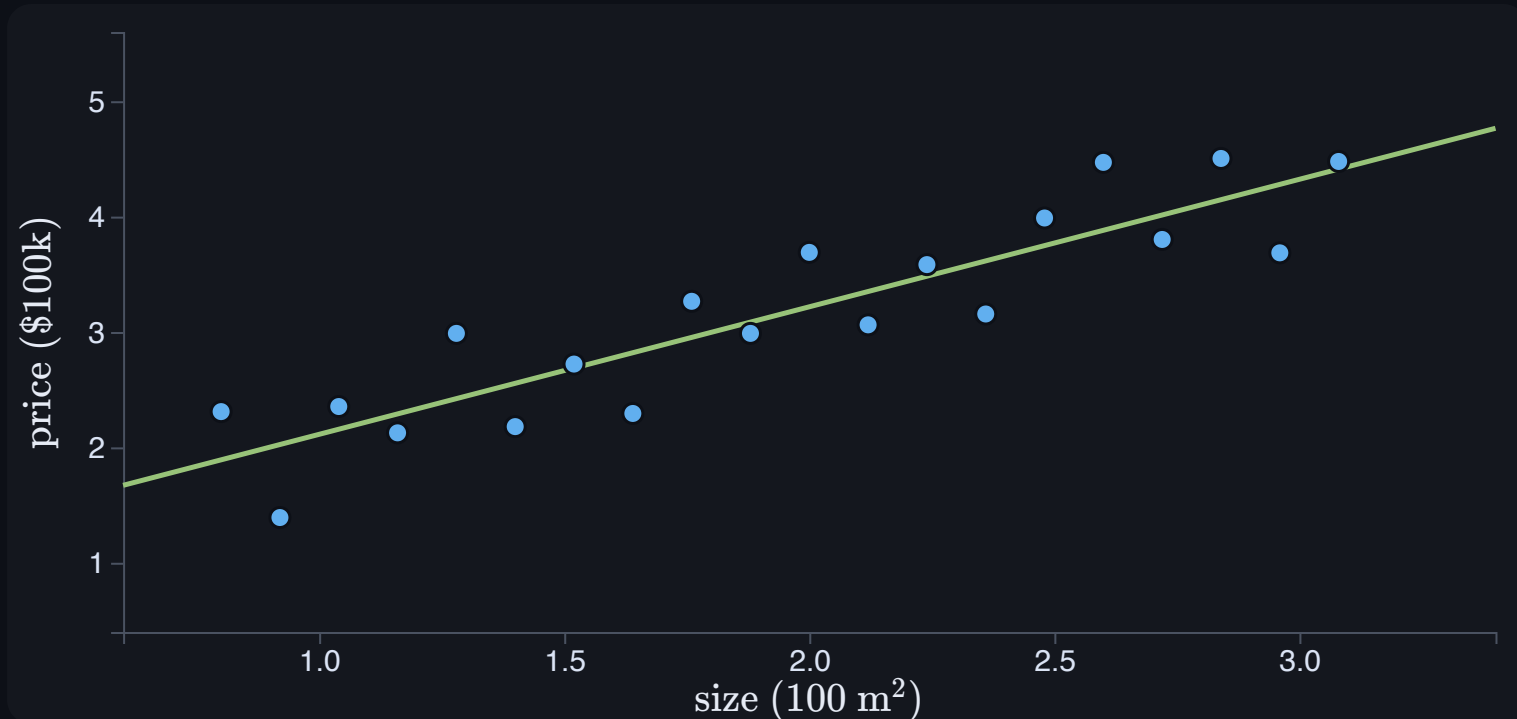
MODEL CLASS

Which function? Linear, polynomial, a neural network? Choosing the family of functions is the first decision.

PARAMETERS

Within a class, θ selects one specific function. Learning is choosing θ from data.

Which Function Should We Choose?



Linear

Polynomial

k-NN average

$$P = 1.11s + 1.01$$

A higher-degree polynomial fits the training points better. Is it a better model? (Lecture 5: generalization)

The Linear Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$$

With the convention $x_0 = 1$, this is a single inner product:

THE LINEAR MODEL

$$h_{\theta}(x) = \theta^{\top} x, \quad \theta \in \mathbb{R}^{d+1}$$

$$h_{\theta}(x) = \underbrace{[\theta_0, \theta_1, \dots, \theta_d]}_{\text{unknown parameters}} \cdot \underbrace{[1, x_1, \dots, x_d]}_{\text{input features}}$$

INTERCEPT

θ_0 ("bias") is the prediction when all inputs are zero.

SENSITIVITY

θ_j is how much $\hat{y}$ changes per unit of x_j .

The Cost Function

Make "best fit" precise. Each **residual** is $h_{\theta}(x^{(i)}) - y^{(i)}$: a square of that area.

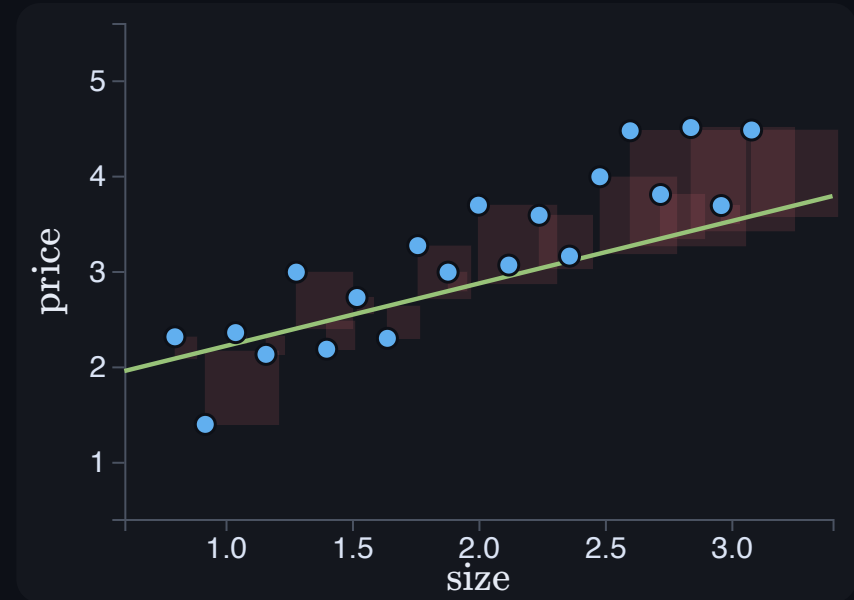
$$J(\theta) = \frac{1}{2} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

WHY SQUARE?

Penalizes big misses, treats both signs alike, and is smooth: calculus works.

WHY ONE HALF?

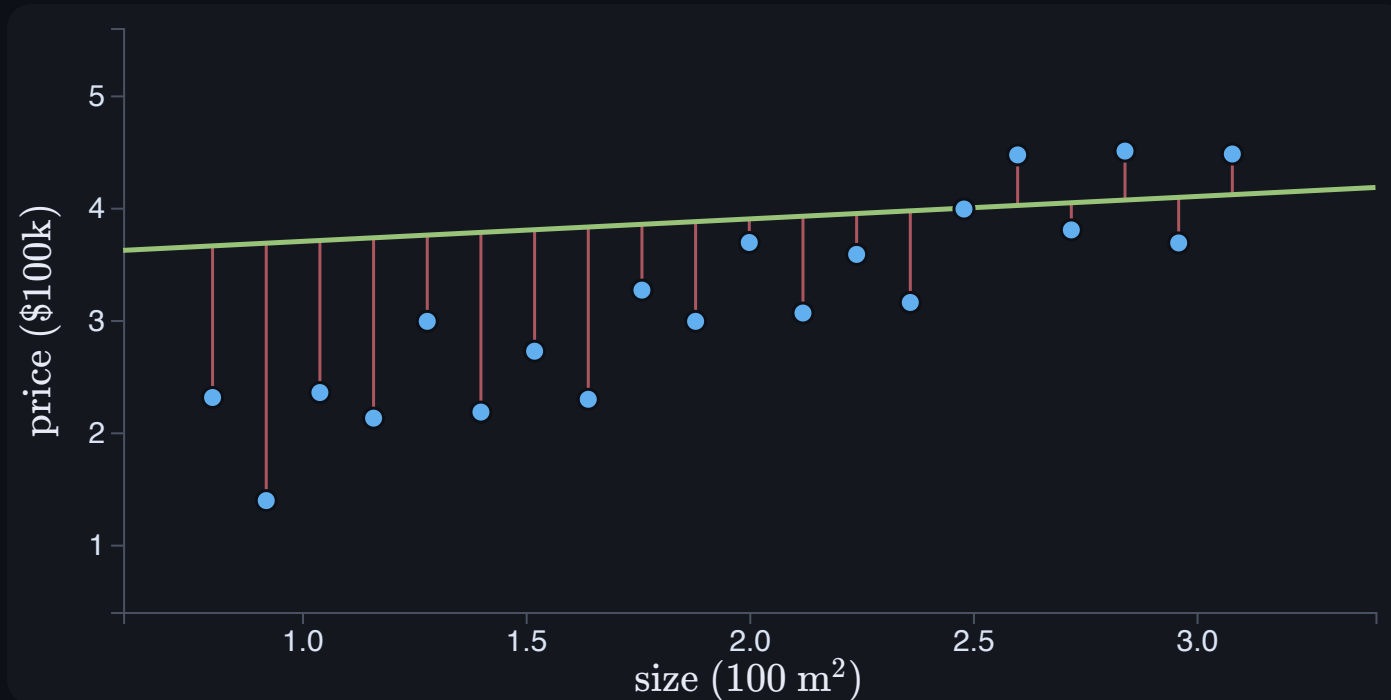
Pure convenience: it cancels when we differentiate.



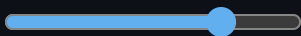
the squares whose total area J measures

Learning becomes an **optimization problem**: $\hat{\theta} = \arg \min_{\theta} J(\theta)$. Next: fit it yourself and watch J respond.

Fit the Line Yourself



θ_0



3.50

θ_1



0.20

Show squared errors

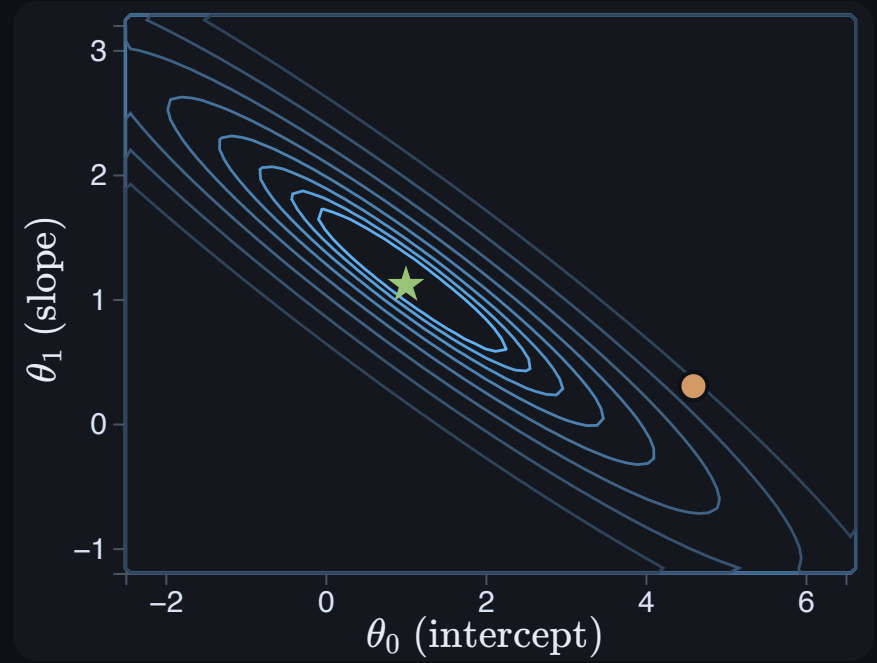
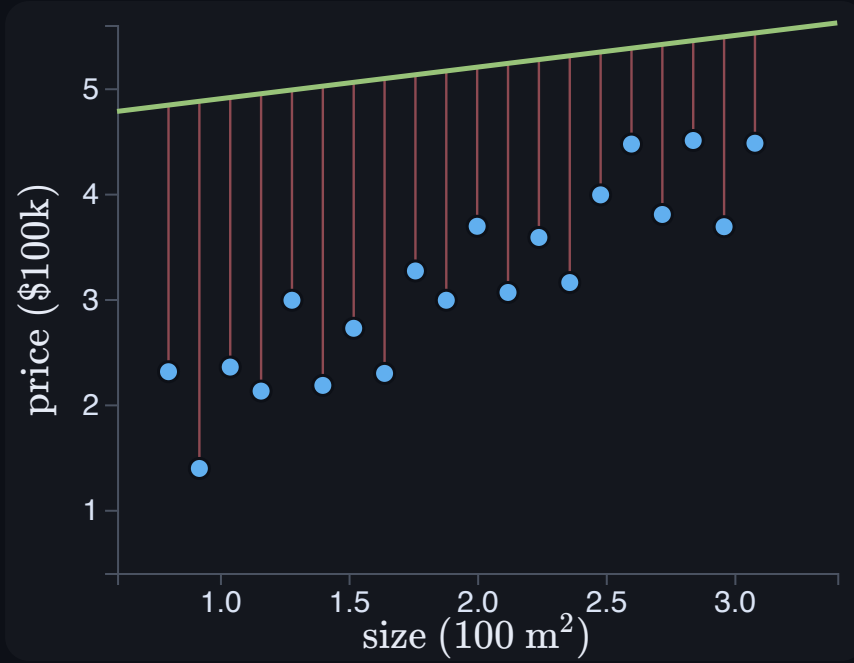
Best fit

MSE: 1.081

The readout is the cost $J(\theta)$ you just defined (scaled): drag θ_0 , θ_1 and watch it respond.

The Loss Landscape

Every point in (θ_0, θ_1) -space is a candidate line. **Drag the orange point.**



Left: data and the line for the current θ . Right: contours of $J(\theta)$; the minimum ★ is the least-squares solution.

Gradient Descent

Can't see the whole landscape? Feel the slope under your feet and walk downhill.

UPDATE RULE

`while` not converged:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

DIRECTION

$\partial J / \partial \theta_j$ points uphill, so we step against it.

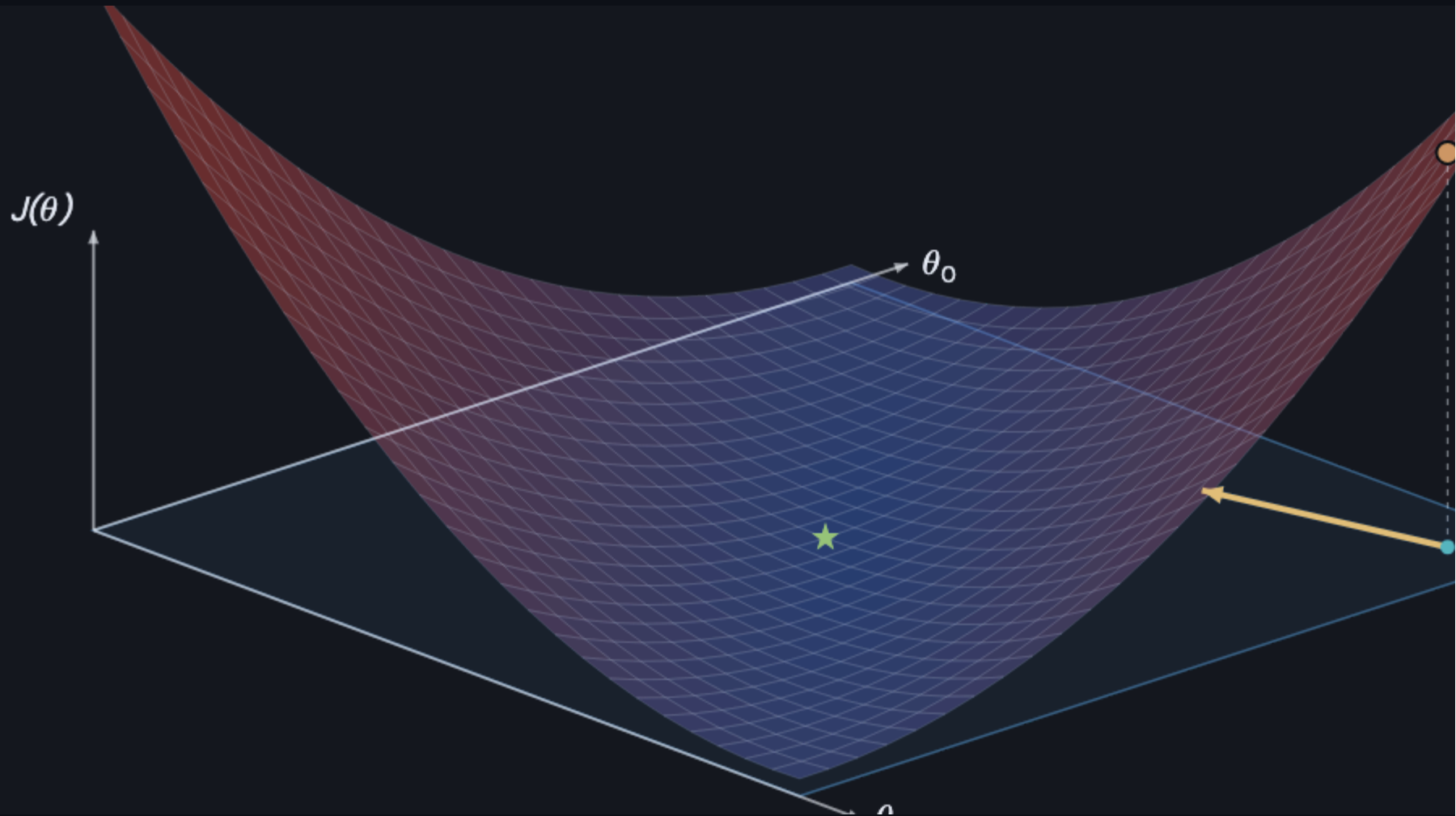
LEARNING RATE

α sets the step size: the most important knob in all of ML.

next: the gradient, worked out by hand



The Loss Surface



Step

▶ Play

Reset

drag to rotate

$\theta := \theta - \alpha \nabla J$ on the floor, step by step

Computing the Gradient

For one example (x, y) (drop the sum for a moment):

$$\frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 = (h_\theta(x) - y) \cdot \frac{\partial}{\partial \theta_j} h_\theta(x)$$

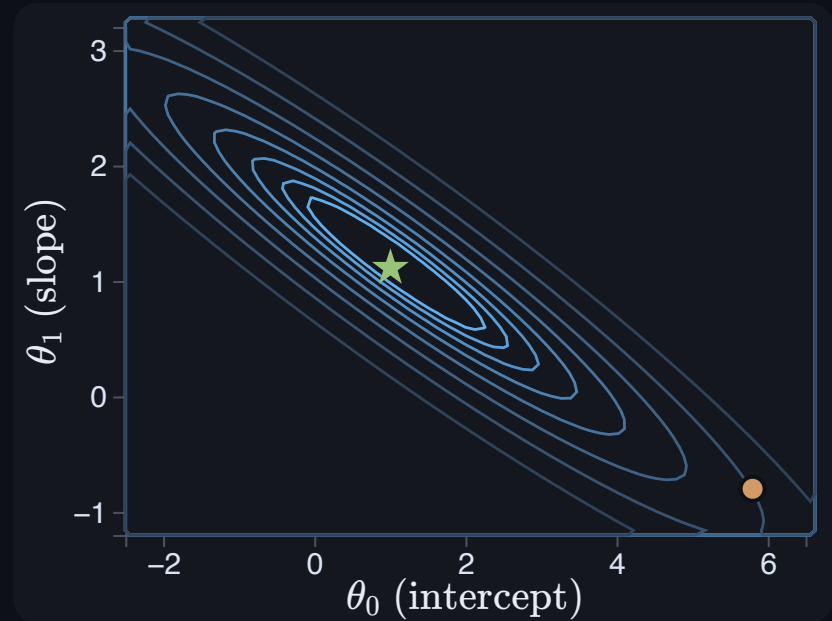
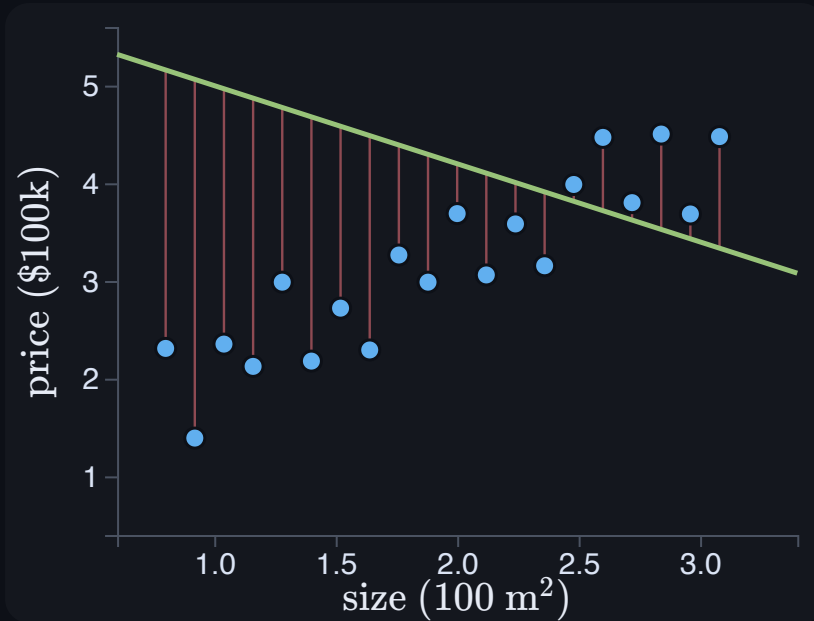
$$\frac{\partial}{\partial \theta_j} h_\theta(x) = \frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \cdots + \theta_d x_d) = x_j$$

LMS UPDATE

$$\theta_j := \theta_j - \alpha \underbrace{(h_\theta(x) - y) x_j}_{\partial J / \partial \theta_j}$$

This per-example update is the **LMS rule** (Widrow–Hoff, 1960).

Watch It Learn



Step

▶ Play

Reset

α



0.016

Stochastic: off

iter 0

· J = 30.86

Try a large α and watch it overshoot and diverge. Turn on stochastic mode to see the noisy path of SGD.

Gradient Descent in Ten Lines

```
1 import numpy as np
2
3 X = np.c_[np.ones(20), x]    # 20 x 2
4 theta = np.zeros(2)
5
6 def grad(theta):
7     # grad J = X.T (X theta - y)
8     return X.T @ (X @ theta - y)
9
10 alpha = 0.01
11 for t in range(200):
12     theta -= alpha * grad(theta)
```

`X.shape = (20, 2)`

$$X = \begin{bmatrix} 1.00 & 0.80 \\ 1.00 & 0.92 \\ 1.00 & 1.04 \\ \vdots & \vdots \end{bmatrix}$$
$$\theta = [0, 0]^T$$

Outputs on the right are computed **live from this data** as the highlight advances. SGD swaps `grad` for one random example: cheaper, noisier, same destination.

Gradient Descent in Ten Lines

```
1 import numpy as np
2
3 X = np.c_[np.ones(20), x]    # 20 x 2
4 theta = np.zeros(2)
5
6 def grad(theta):
7     # grad J = X.T (X theta - y)
8     return X.T @ (X @ theta - y)
9
10 alpha = 0.01
11 for t in range(200):
12     theta -= alpha * grad(theta)
```

`X.shape = (20, 2)`

$$X = \begin{bmatrix} 1.00 & 0.80 \\ 1.00 & 0.92 \\ 1.00 & 1.04 \\ \vdots & \vdots \end{bmatrix}$$
$$\theta = [0, 0]^T$$

Outputs on the right are computed **live from this data** as the highlight advances. SGD swaps `grad` for one random example: cheaper, noisier, same destination.

Gradient Descent in Ten Lines

```
1 import numpy as np
2
3 X = np.c_[np.ones(20), x]    # 20 x 2
4 theta = np.zeros(2)
5
6 def grad(theta):
7     # grad J = X.T (X theta - y)
8     return X.T @ (X @ theta - y)
9
10 alpha = 0.01
11 for t in range(200):
12     theta -= alpha * grad(theta)
```

`X.shape = (20, 2)`

$$X = \begin{bmatrix} 1.00 & 0.80 \\ 1.00 & 0.92 \\ 1.00 & 1.04 \\ \vdots & \vdots \end{bmatrix}$$
$$\theta = [0, 0]^T$$

Outputs on the right are computed **live from this data** as the highlight advances. SGD swaps `grad` for one random example: cheaper, noisier, same destination.

Can We Skip the Iteration?

Stack the data into a matrix, one example per row:

$$X = \begin{bmatrix} \text{---} (x^{(1)})^\top \text{---} \\ \vdots \\ \text{---} (x^{(n)})^\top \text{---} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

Then the whole cost is one squared distance:

$$J(\theta) = \frac{1}{2} \|X\theta - \mathbf{y}\|^2$$

EXERCISE

Set $\nabla_{\theta} J(\theta) = 0$ and solve for θ .

One line of matrix calculus. Try it before the next slide.

The Normal Equations

$$\nabla_{\theta} J = \nabla_{\theta} \frac{1}{2} (X\theta - \mathbf{y})^{\top} (X\theta - \mathbf{y}) = X^{\top} (X\theta - \mathbf{y})$$

Set it to zero: $X^{\top} X\theta = X^{\top} \mathbf{y}$ (the **normal equations**)

CLOSED FORM

$$\hat{\theta} = (X^{\top} X)^{-1} X^{\top} \mathbf{y}$$

Full derivation with the matrix-calculus rules: [CS229 notes, Part I](#).

Gradient Descent vs. Normal Equations

NORMAL EQUATIONS

- Exact: no α , no iterations
- Needs $(X^\top X)^{-1}$: $O(d^3)$
- Fine for d up to a few thousand

GRADIENT DESCENT

- Iterative, needs α
- Each step $O(nd)$, scales to huge d
- The *only* option for neural networks

linear regression: one bowl, one bottom



a neural network loss: which valley?



TAKEAWAY

Linear regression is the rare case: one bowl, one bottom, a closed form. Everything else in this course descends a jagged landscape. **Neural networks?**

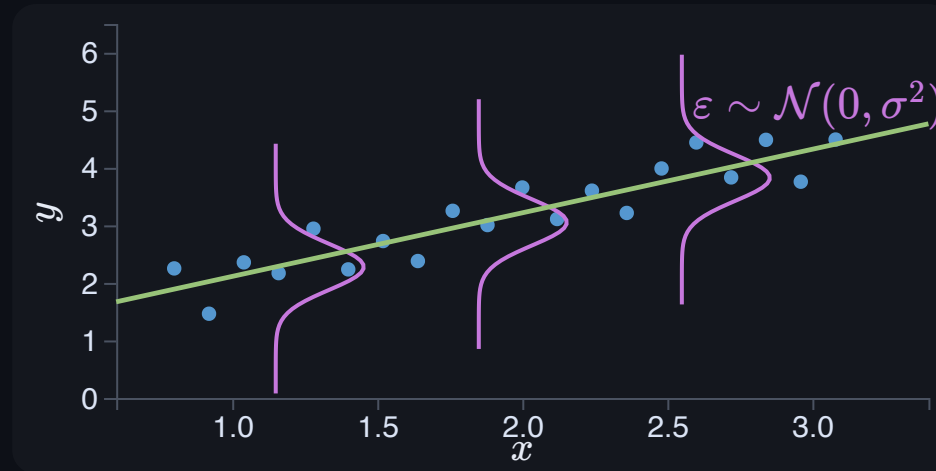
Why Least Squares? A Probabilistic View

Assume outputs are the model *plus Gaussian noise*:

$$y^{(i)} = \theta^\top x^{(i)} + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2) \text{ i.i.d.}$$



the model is the line;
every step wobbles
around it



noise σ  0.40

each $y^{(i)}$ is a draw from a Gaussian centered
on the line

How probable
is the observed
dataset, as a
function of θ ?
That is the
likelihood.

The Likelihood of the Data

The noise model says each output, given its input, is Gaussian around the line:

$$p(y^{(i)} \mid x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2}\right)$$

The noise draws are **independent**, so the probability of the whole dataset is the product:

$$L(\theta) = \prod_{i=1}^n p(y^{(i)} \mid x^{(i)}; \theta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2}\right)$$

— MAXIMUM LIKELIHOOD —

Pick the θ that makes the observed data most probable.

Least Squares Is Maximum Likelihood

Products are awkward. Take the log, and the product becomes a sum:

$$\log L(\theta) = \sum_{i=1}^n \log \left[\frac{1}{\sqrt{2\pi} \sigma} \exp \left(-\frac{(y^{(i)} - \theta^\top x^{(i)})^2}{2\sigma^2} \right) \right]$$

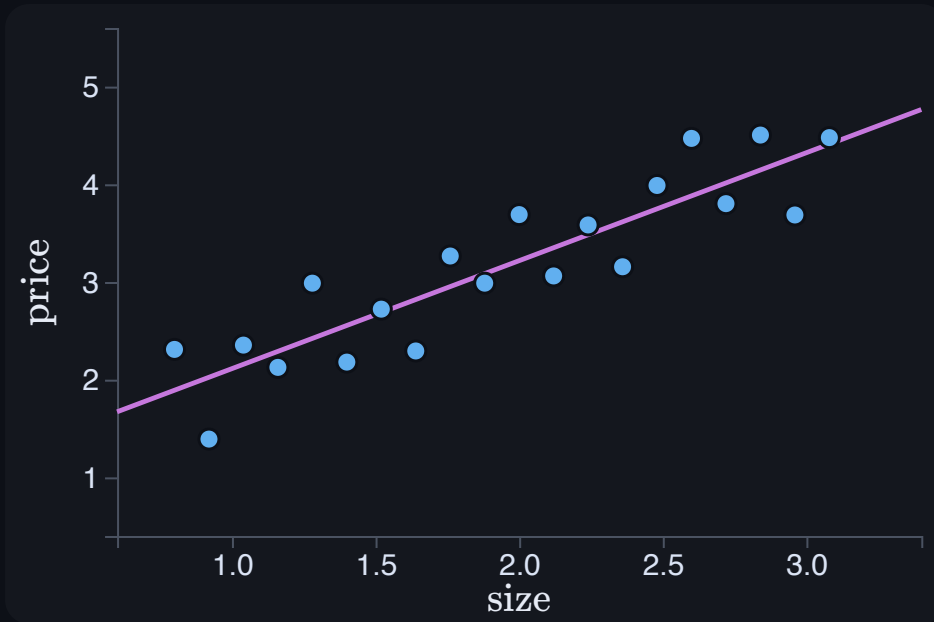
$$= n \log \frac{1}{\sqrt{2\pi} \sigma} - \frac{1}{\sigma^2} \cdot \underbrace{\frac{1}{2} \sum_{i=1}^n (y^{(i)} - \theta^\top x^{(i)})^2}_{J(\theta)}$$

THE PUNCHLINE

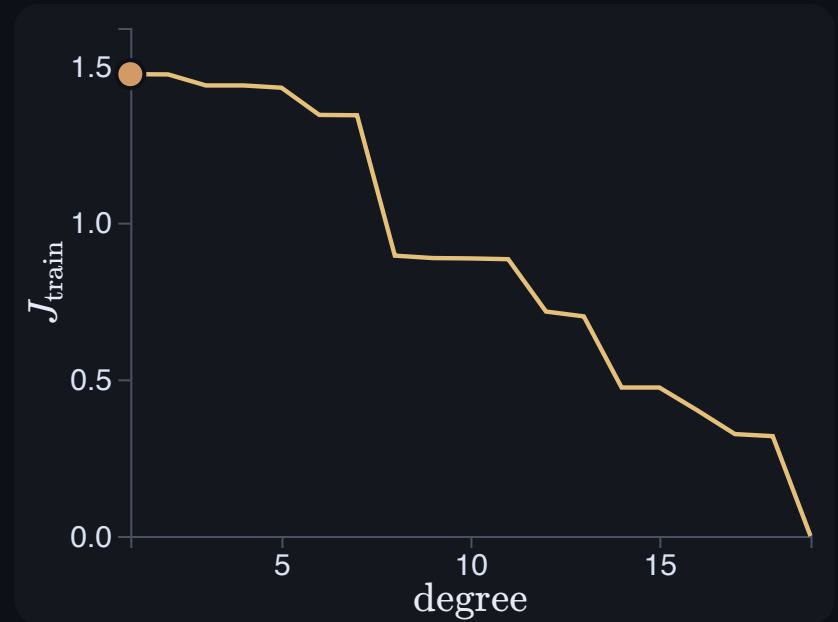
Maximizing $\log L$ is minimizing $J(\theta)$: the noise model chooses the loss.

Different noise, different loss: Laplacian noise gives absolute error (L1), robust to outliers.

Why Not Interpolate *All* the Points?



degree 1



training error: 1.474

At degree 19 the curve passes through every point: **zero training error**. Do you trust it between the points? Next lecture: **generalization**.

Summary

THE MODEL

$$h_{\theta}(x) = \theta^{\top} x$$

linear in the parameters

THE OBJECTIVE

$$J(\theta) = \frac{1}{2} \sum_i (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

squared error = Gaussian maximum
likelihood

SOLVER 1: NORMAL EQUATIONS

$$\hat{\theta} = (X^{\top} X)^{-1} X^{\top} \mathbf{y}$$

exact, when d is small

SOLVER 2: GRADIENT DESCENT

$$\theta := \theta - \alpha \nabla J$$

iterative, scales; α too big diverges, too
small crawls

Next lecture: features, overfitting, and **generalization**: when a better fit is a worse model.